AI-Powered Escape Room

1. Introduction

The AI-Powered Escape Room is an interactive game developed using Python, Streamlit and Google's Gemini AI. The player explores a mysterious library, interacts with different objects, solves puzzles, collects items and attempts to escape.

The project demonstrates how a Large Language Model (LLM) can be integrated into an interactive application. It also explores the importance of controlling AI-generated actions using traditional Python programming logic.

2. Project Objectives

The main objectives of this project are:

- Develop an interactive escape-room game using Python and Streamlit.
- Integrate Gemini AI as the Game Master.
- Allow players to interact with the game using natural language.
- Maintain game progress, inventory and puzzle information during a session.
- Improve reliability by introducing structured game state and Python-based action validation.
- Compare a conversational AI-based approach with a more controlled AI application architecture.

3. Technologies Used

Technology	**Purpose**
Python	Main programming language and game logic
Streamlit	User interface
Google Gemini API	Natural-language understanding and game narration
Python dictionaries	Structured game state and inventory
Session State	Maintaining player progress during gameplay
dotenv	Managing API keys securely

4. Version 1: Conversational AI Escape Room

Version 1 was developed as the initial implementation of the escape-room game.

The player explores a mysterious library containing five interactive objects: Desk, Bookshelf, Painting, Locked Drawer and Exit Door.

The player interacts with the objects, solves three puzzles, collects a Brass Key, unlocks the drawer, discovers a Secret Note and finally attempts to escape.

Gemini acts as the Game Master. It interprets natural-language messages and provides responses to the player. Python handles game actions and maintains session progress.

Version 1 demonstrates how an LLM can make a traditional game more interactive through natural-language communication.

5. Version 2: Structured and Validated Escape Room

Version 2 improves the application by introducing a more structured approach to game management.

The game uses a structured Python dictionary to maintain information such as:

- Player inventory
- Puzzle completion status
- Score
- Game progress
- Drawer and exit conditions
- Other session-specific game information

Dedicated Python functions validate important actions, including collecting and using items, solving puzzles, requesting hints, unlocking the drawer, reading the Secret Note and unlocking the exit.

Gemini is used to interpret the player's request and generate narration. However, Python determines whether an action is valid and whether the game state should change.

This approach makes important game rules independent of AI-generated responses.

6. Main Improvements from Version 1 to Version 2

Version 1 focuses primarily on conversational interaction, while Version 2 introduces stronger separation between AI interpretation and Python game logic.

The major improvements include:

- Structured game state management.
- Dedicated Python action validators.
- Centralized inventory and puzzle management.
- Validation of incorrect and repeated actions.
- Better control over score and victory conditions.
- Improved handling of adversarial requests, such as attempts to modify the score or claim victory without completing the game.
- Randomized puzzle challenges and codes within a game session.

7. Testing and Evaluation

Both versions were tested using normal gameplay actions and challenging scenarios.

The tests included:

- Collecting and using the Brass Key.
- Solving puzzles and requesting hints.
- Attempting to collect an item more than once.
- Using an item without collecting it.
- Entering incorrect drawer and exit codes.
- Attempting to modify the score through natural-language instructions.
- Attempting to claim victory without satisfying the game conditions.
- Restarting the game and checking whether progress resets.

The tests demonstrated that Version 2 provides a more structured approach to validating actions and controlling game progress.

Detailed test scenarios and their results are documented separately in the Test-Cases folder.

8. Conclusion

The AI-Powered Escape Room demonstrates the integration of conversational AI into an interactive Python application.

Version 1 establishes the basic conversational game experience. Version 2 improves the architecture by separating natural-language interpretation, Python validation and game-state management.

An important lesson from this project is that an LLM should not be the sole authority for important application decisions. Python validation provides greater control over game rules, inventory, puzzle completion and victory conditions.

The project provided practical experience with Python, Streamlit, Gemini API integration, structured data management, application architecture and testing.

9. Limitations and Future Improvements

The current application maintains game progress within the active Streamlit session rather than using permanent database storage.

Future improvements could include:

- Persistent player accounts and saved game progress.
- Additional rooms and puzzle categories.
- More detailed gameplay statistics.
- Improved automated testing.
- Additional validation for unusual or ambiguous player requests.